ФЕДЕРАЛЬНОЕ АГЕНТСТВО ПО ОБРАЗОВАНИЮ

ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ «САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ имени академика С.П. КОРОЛЁВА»

КАФЕДРА «ТЕХНИЧЕСКАЯ КИБЕРНЕТИКА»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Объектно-ориентированное программирование»

ЛАБОРАТОРНАЯ РАБОТА №3

Студент Ефремова А. С.

Группа 6301-030301D

Руководитель Борисов Д. С.

Оценка _____

САМАРА 2025

Задание 2

Созданы два специализированных класса исключений: FunctionPointIndexOutOfBoundsException для обработки неверных индексов и InappropriateFunctionPointException для нарушений упорядоченности точек.

```java
package functions;

public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException {
    public FunctionPointIndexOutOfBoundsException() {//исключение без специального сообщения
        super();//констр.родителя
    }

    public FunctionPointIndexOutOfBoundsException(String message) {//когда нужно указать конк
        super(message);
    }
}
```

```java
package functions;

public class InappropriateFunctionPointException extends Exception {
    public InappropriateFunctionPointException() {//создает искл.без
        super(); //вызывает конструктор родителя(Exception)
    }

    public InappropriateFunctionPointException(String message) {//ис
        super(message);
    }
}
```

Задание 3

Доработан класс TabulatedFunction (переименованный позже в ArrayTabulatedFunction) с добавлением проверок и выбрасыванием исключений при некорректных операциях.

```java
package functions;

public class ArrayTabulatedFunction implements TabulatedFunction {
    private FunctionPoint[] points; //массив для хранения точек
    private int pointsCount;        //счетчик кол-ва точек (может быть меньше длины массива!)
    private static final double EPSILON = 1e-10; // для сравнения double чисел

    //Конструктор1(точки от leftX до rightX, y=0)
    public ArrayTabulatedFunction(double leftX, double rightX, int pointsCount) {
        //проверка параметров конструктора
        if (leftX >= rightX - EPSILON) {
            throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой");
        }
        if (pointsCount < 2) {
            throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
        }

        //создаем массив
        this.points = new FunctionPoint[pointsCount];
        this.pointsCount = pointsCount;

        //вычисляем расстояние между точками
        double move=(rightX - leftX) / (pointsCount - 1);

        //создаем точки с равными интервалами
        for (int i = 0;i < pointsCount;i++) {
            double x = leftX + i * move;
            points[i] = new FunctionPoint(x,y: 0.0);
        }
    }

    //Конструктор2(точки от leftX до rightX, y из values)
    public ArrayTabulatedFunction(double leftX, double rightX, double[] values) {
        //проверка параметров конструктора
        if (leftX >= rightX - EPSILON) {
            throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой");
        }
        if (values.length < 2) {
            throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
        }

        //создаем массив
        this.points = new FunctionPoint[values.length];
        this.pointsCount = values.length;

        //вычисляем расстояние между точками
        double move = (rightX - leftX) / (pointsCount - 1);

        //создаем точки с заданными значениями y
        for (int i = 0; i < pointsCount;i++) {
            double x = leftX + i * move;
            points[i] = new FunctionPoint(x, values[i]);
        }
    }
```

```java
@Override
public double getLeftDomainBorder() {
    return points[0].getX(); //первая точка определяет левую границу
}
//метод для правой границы области определения табулированной ф-ии
@Override
public double getRightDomainBorder() {
    return points[pointsCount - 1].getX(); //последняя точка - правую границу
}
//метод для количества точек
@Override
public int getPointsCount() {
    return pointsCount;
}

//метод для точки по индексу(возвращаем копию)
@Override
public FunctionPoint getPoint(int index) {
    //проверка индекса
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " вне границ [0, " + (pointsCount-1) + "]");
    }
    return new FunctionPoint(points[index]); //возвращаем копию для защиты данных
}

//замена точки
@Override
public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
    //проверка индекса
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " вне границ [0, " + (pointsCount-1) + "]");
    }

    //создаем копию (защаем от изменений )
    FunctionPoint newPoint = new FunctionPoint(point);

    //проверка чтобы не сломался порядок по возрастанию x
    // если x новой точки <= предыдущей -> исключение
    if (index > 0 && newPoint.getX() - points[index-1].getX() <= EPSILON) {
        throw new InappropriateFunctionPointException(message: "X новой точки должен быть больше предыдущего");
    }
    // если x новой точки >= следуующей -> исключение
    if (index < pointsCount-1 && points[index+1].getX() - newPoint.getX() <= EPSILON) {
        throw new InappropriateFunctionPointException(message: "X новой точки должен быть меньше следующего");
    }

    points[index] = newPoint;
}
```

Задание 4

Создан класс LinkedListTabulatedFunction с двусвязным циклическим списком для хранения точек, включая внутренний класс Node и методы работы со списком.

```java
package functions;

public class LinkedListTabulatedFunction implements TabulatedFunction {
    //внутренний класс для узла списка
    private class FunctionNode {
        FunctionPoint point;  //данные точки
        FunctionNode prev;    //ссылка на предыдущий узел
        FunctionNode next;    //ссылка на следующий узел

        //конструктор узла
        FunctionNode(FunctionPoint point, FunctionNode prev, FunctionNode next) {
            this.point = point;
            this.prev = prev;
            this.next = next;
        }
    }
    //поля класса
    private FunctionNode head;       //голова списка (не содержит данных)
    private int pointsCount;         //кол-во точек
    private static final double EPSILON = 1e-10;

    //вспомогательные поля для оптимизации доступа
    private FunctionNode lastNode;   //последний используемый узел
    private int lastIndex;           //индекс последнего используемого узла
```

```java
private FunctionNode getNodeByIndex(int index) {
    //проверка индекса
    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс " + index + " вне границ [0, " + (pointsCount-1) + "]");
    }

    //проверяем можно ли использовать lastNode для оптимизации
    if (lastIndex != -1) {
        int diff = Math.abs(index - lastIndex);
        //если ближе идти от lastNode чем от головы
        if (diff < index && diff < pointsCount - index) {
            FunctionNode current = lastNode;
            if (index > lastIndex) {
                //двигаемся вперед
                for (int i = lastIndex; i < index; i++) {
                    current = current.next;
                }
            } else {
                //двигаемся назад
                for (int i = lastIndex; i > index; i--) {
                    current = current.prev;
                }
            }
            lastNode = current; //запоминаем для следующих вызовов
            lastIndex = index;
            return current;
        }
    }

    //иначе идем от головы
    FunctionNode current = head.next;
    for (int i = 0; i < index; i++) {
        current = current.next;
    }
    lastNode = current;
    lastIndex = index;
    return current;
}
```

Задание 5

Класс LinkedListTabulatedFunction дополнен методами интерфейса, обеспечивающими работу с табулированной функцией, включая конструкторы, методы доступа к точкам и вычисление значений.

```java
public LinkedListTabulatedFunction(double leftX, double rightX, int pointsCount) {
    //проверка параметров конструктора
    if (leftX >= rightX - EPSILON) {
        throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой");
    }
    if (pointsCount < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
    }

    initList(); //создаем пустой список
    double move = (rightX - leftX) / (pointsCount - 1);

    //создаем точки
    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * move;
    //addNodeToTail() создает новый узел и добавляет в конец
        addNodeToTail().point = new FunctionPoint(x, y: 0.0);
    }
}

//Конструктор2(с заданными У)
public LinkedListTabulatedFunction(double leftX, double rightX, double[] values) {
    //проверка параметров конструктора
    if (leftX >= rightX - EPSILON) {
        throw new IllegalArgumentException(s: "Левая граница должна быть меньше правой");
    }
    if (values.length < 2) {
        throw new IllegalArgumentException(s: "Количество точек должно быть не менее 2");
    }

    initList(); //создаем пустой список
    double move = (rightX - leftX) / (values.length - 1);

    //создаем точки с заданными значениями у
    for (int i = 0; i < values.length; i++) {
        double x = leftX + i * move;
        addNodeToTail().point = new FunctionPoint(x, values[i]);
    }
}
```

```java
@Override
public double getFunctionValue(double x) {//если список пуст или ошибка границы
    if (pointsCount == 0 || x < getLeftDomainBorder() - EPSILON || x > getRightDomainBorder() + EPSILON) {
        return Double.NaN;
    }

    //ищем отрезок где x
    FunctionNode current = head.next;//первая точка
    for (int i = 0; i < pointsCount - 1; i++) {
        double x1 = current.point.getX();
        double x2 = current.next.point.getX();
        double y1 = current.point.getY();
        double y2 = current.next.point.getY();

        //если x совпадает с x1 (в пределах эпсилон)
        if (Math.abs(x - x1) < EPSILON) {
            return y1; //возвращ соотв. У
        }

        //если x совпадает с x2 (в пределах эпсилон)
        if (Math.abs(x - x2) < EPSILON) {
            return y2; //возвращ соотв. У
        }

        //если x в текущем отрезке
        if (x >= x1 - EPSILON && x <= x2 + EPSILON) {
            //лин. интерполяция
            return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
        }
        current = current.next;
    }
```

Задание 6

Создан интерфейс TabulatedFunction и переименован TabulatedFunction в ArrayTabulatedFunction, оба класса теперь реализуют единый интерфейс.

```java
package functions;

public interface TabulatedFunction {//шаблон
    double getLeftDomainBorder();//возвр. левую границу

    double getRightDomainBorder();//возвр. правую границу

    int getPointsCount();//возвр. кол-во точек

    FunctionPoint getPoint(int index);//возвр. точку к индексу

    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException;//

    double getPointX(int index);//возвр. X коорд.точки

    void setPointX(int index, double x) throws InappropriateFunctionPointException;//изменяет

    double getPointY(int index);//возвр. Y коорд.точки

    void setPointY(int index, double y);//изменяет только Y коор. точки

    void deletePoint(int index);//удаляет точку по индексу

    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException;//добавляет

    double getFunctionValue(double x);//вычисляет знач. ф-ии в заданной точке x
}
```

Задание 7

Изменена тестовая программа Main, подтверждающая корректную работу обоих классов, обработку исключений и взаимозаменяемость реализаций через интерфейс.

```java
import functions.*;

public class Main {
    private static void printPoints(TabulatedFunction func) {
        System.out.print(s: "Точки: [");
        for (int i = 0; i < func.getPointsCount(); i++) {
            System.out.print("(" + func.getPointX(i) + ", " + func.getPointY(i) + ")");
            if (i < func.getPointsCount() - 1) {
                System.out.print(s: ", ");
            }
        }
        System.out.println(x: "]");
    }

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        double[] values = {4, 1, 0, 1, 4, 9};

        System.out.println(x: " Тестирование ArrayTabulatedFunction");
        testFunction(new ArrayTabulatedFunction(-2, rightX: 3, values));

        System.out.println(x: "\n Тестирование LinkedListTabulatedFunction");
        testFunction(new LinkedListTabulatedFunction(-2, rightX: 3, values));

        System.out.println(x: "\n Проверка исключений ");
        testExceptions();
    }

    private static void testFunction(TabulatedFunction func) {
        System.out.println(x: "Исходная функция");
        System.out.println("Левая граница: " + func.getLeftDomainBorder());
        System.out.println("Правая граница: " + func.getRightDomainBorder());
        System.out.println("Кол-во точек: " + func.getPointsCount());
        printPoints(func);

        System.out.println(x: " Проверка значений функции");
        double[] testPoints = {-3, -2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5, 2, 2.5, 3, 4};
        for (double x : testPoints) {
            double result = func.getFunctionValue(x);
            System.out.println("f(" + x + ") = " + result);
        }

        System.out.println(x: "\n Проверка операций с точками");

        System.out.println(x: "1. Добавляем точку (-0.7, 0.49):");
        try {
            func.addPoint(new FunctionPoint(-0.7, y: 0.49));
            printPoints(func);
            System.out.println("f(-0.7) = " + func.getFunctionValue(-0.7));
        } catch (InappropriateFunctionPointException e) {
            System.out.println("Ошибка при добавлении: " + e.getMessage());
        }

        System.out.println(x: "\n2. Удаляем точку с индексом 1:");
        try {
            func.deletePoint(index: 1);
            printPoints(func);
        } catch (Exception e) {
            System.out.println("Ошибка при удалении: " + e.getMessage());
        }
```

```
    System.out.println(x: "\n3. Изменяем у точки с индексом 2 на 0.5:");
    try {
        func.setPointY(index: 2, y: 0.5);
        printPoints(func);
    } catch (Exception e) {
        System.out.println("Ошибка при изменении: " + e.getMessage());
    }

    System.out.println(x: "\n Итоговый результат");
    System.out.println("f(-0.7) = " + func.getFunctionValue(-0.7));
    printPoints(func);
}

private static void testExceptions() {
    System.out.println(x: "1. Неправильные границы функции (левая >= правой):");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(leftX: 3, -2, new double[]{1, 2, 3});
        System.out.println(x: "   Ошибка: должно было быть исключение!");
    } catch (IllegalArgumentException e) {
        System.out.println("   Получено: " + e.getMessage());
    }

    System.out.println(x: "2. Слишком мало точек (меньше 2):");
    try {
        TabulatedFunction func = new LinkedListTabulatedFunction(-2, rightX: 3, new double[]{4});
        System.out.println(x: "   Ошибка: должно было быть исключение!");
    } catch (IllegalArgumentException e) {
        System.out.println("   Получено: " + e.getMessage());
    }

    System.out.println(x: "3. Несуществующий индекс точки:");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(-2, rightX: 3, new double[]{4, 1, 0, 1, 4, 9});
        func.getPoint(index: 10);
        System.out.println(x: "   Ошибка: должно было быть исключение!");
    } catch (FunctionPointIndexOutOfBoundsException e) {
        System.out.println("   Получено: " + e.getMessage());
    }

    System.out.println(x: "4. Нарушение порядка точек по X:");
    try {
        TabulatedFunction func = new LinkedListTabulatedFunction(-2, rightX: 3, pointsCount: 6);
        func.setPointX(index: 2, x: 5.0);
        System.out.println(x: "   Ошибка: должно было быть исключение!");
    } catch (InappropriateFunctionPointException e) {
        System.out.println("   Получено: " + e.getMessage());
    }

    System.out.println(x: "5. Добавление точки с уже существующим X:");
    try {
        TabulatedFunction func = new ArrayTabulatedFunction(-2, rightX: 3, new double[]{4, 1, 0, 1, 4, 9});
        func.addPoint(new FunctionPoint(x: 1.0, y: 5.0));
        System.out.println(x: "   Ошибка: должно было быть исключение!");
    } catch (InappropriateFunctionPointException e) {
        System.out.println("   Получено: " + e.getMessage());
    }
```

```
        System.out.println(x: "6. Удаление точки когда их останется слишком мало:");
        try {
            TabulatedFunction func = new LinkedListTabulatedFunction(-2, rightX: 3, new double[]{4, 1, 0});
            func.deletePoint(index: 0);
            System.out.println(x: "   Ошибка: должно было быть исключение!");
        } catch (IllegalStateException e) {
            System.out.println("   Получено: " + e.getMessage());
        }
    }
}
```

Реализация:

```
 Тестирование ArrayTabulatedFunction
Исходная функция
Левая граница: -2.0
Правая граница: 3.0
Кол-во точек: 6
Точки: [(-2.0, 4.0), (-1.0, 1.0), (0.0, 0.0), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]
 Проверка значений функции
f(-3.0) = NaN
f(-2.0) = 4.0
f(-1.5) = 2.5
f(-1.0) = 1.0
f(-0.5) = 0.5
f(0.0) = 0.0
f(0.5) = 0.5
f(1.0) = 1.0
f(1.5) = 2.5
f(2.0) = 4.0
f(2.5) = 6.5
f(3.0) = 9.0
f(4.0) = NaN

 Проверка операций с точками
1. Добавляем точку (-0.7, 0.49):
Точки: [(-2.0, 4.0), (-1.0, 1.0), (-0.7, 0.49), (0.0, 0.0), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]
f(-0.7) = 0.49

2. Удаляем точку с индексом 1:
Точки: [(-2.0, 4.0), (-0.7, 0.49), (0.0, 0.0), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]

3. Изменяем у точки с индексом 2 на 0.5:
Точки: [(-2.0, 4.0), (-0.7, 0.49), (0.0, 0.5), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]

 Итоговый результат
f(-0.7) = 0.49
Точки: [(-2.0, 4.0), (-0.7, 0.49), (0.0, 0.5), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]
```

```
 Тестирование LinkedListTabulatedFunction
Исходная функция
Левая граница: -2.0
Правая граница: 3.0
Кол-во точек: 6
Точки: [(-2.0, 4.0), (-1.0, 1.0), (0.0, 0.0), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]
 Проверка значений функции
f(-3.0) = NaN
f(-2.0) = 4.0
f(-1.5) = 2.5
f(-1.0) = 1.0
f(-0.5) = 0.5
f(0.0) = 0.0
f(0.5) = 0.5
f(1.0) = 1.0
f(1.5) = 2.5
f(2.0) = 4.0
f(2.5) = 6.5
f(3.0) = 9.0
f(4.0) = NaN

 Проверка операций с точками
1. Добавляем точку (-0.7, 0.49):
Точки: [(-2.0, 4.0), (-1.0, 1.0), (-0.7, 0.49), (0.0, 0.0), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]
f(-0.7) = 0.49

2. Удаляем точку с индексом 1:
Точки: [(-2.0, 4.0), (-0.7, 0.49), (0.0, 0.0), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]

3. Изменяем у точки с индексом 2 на 0.5:
Точки: [(-2.0, 4.0), (-0.7, 0.49), (0.0, 0.5), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]

 Итоговый результат
f(-0.7) = 0.49
Точки: [(-2.0, 4.0), (-0.7, 0.49), (0.0, 0.5), (1.0, 1.0), (2.0, 4.0), (3.0, 9.0)]
```